Estruturas de Dados I

TAD Pilha

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

30 de setembro de 2026

Objetivos da Aula

- Definir o **TAD Pilha** e sua interface.
- Desenvolver o TAD Pilha por meio de uma **implementação sequencial**.
- Desenvolver o TAD Pilha por meio de uma **implementação encadeada**.
- Aplicar a pilha na **verificação de delimitadores** balanceados.
- Aplicar a pilha na **avaliação de expressões**.

Introdução

Motivação

- Em muitas situações, a ordem em que os elementos saem é determinada pela ordem em que entraram.
- Considere uma **pilha** de pratos sobre a bancada:
  - Cada prato lavado é colocado sobre os anteriores;
  - O prato retirado para uso é o de cima.
- O último prato a entrar é o primeiro a sair.
- Nenhum prato do meio pode ser retirado sem antes remover os que estão acima dele.

Regra LIFO

- Estruturas com esse comportamento são chamadas de **LIFO***.
- **Consequências:**
  - Só existe um ponto de acesso, chamado de **topo**;
  - O elemento mais antigo é o menos acessível.
- Não há busca, não há acesso por posição e não há remoção pelo meio.
- Essa limitação torna as operações simples e rápidas.

*Last In, First Out

Aplicações de Pilhas

- **Desfazer:** o editor de texto guarda cada ação executada; o comando desfazer retira a mais recente.
- **Histórico do navegador:** o botão voltar retorna à última página visitada.
- **Chamadas de função:** cada função chamada é empilhada e só é retirada quando termina.
- **Expressões aritméticas:** o que ainda não pode ser resolvido fica guardado até que os dados necessários estejam disponíveis.

Interface

Operações do TAD Pilha

- **Criar:** produzir uma pilha vazia.
- **Destruir:** liberar a pilha e tudo o que ela contém.
- **Empilhar:** inserir um elemento no topo.
- **Desempilhar:** remover o elemento do topo e devolvê-lo.
- **Consultar o topo:** obter o elemento do topo sem removê-lo.
- **Verificar se está vazia.**
- **Informar o tamanho.**

Decisões de Contrato e Tipo dos Elementos

- A criação devolve `NULL` se não houver memória disponível.
- Operações impossíveis, como desempilhar de uma pilha vazia, devolvem `false` e não alteram a pilha.
- O elemento removido é copiado para um endereço fornecido pelo usuário.
- O tipo dos elementos armazenados recebe um nome próprio:

```
typedef int Item;
```

- Para guardar números reais ou registros completos, basta alterar essa única linha.

Arquivo pilha.h

```
#ifndef PILHA_H
#define PILHA_H

#include <stdbool.h>

typedef int Item;
typedef struct pilha Pilha;

Pilha *criarPilha(void);
void destruirPilha(Pilha *p);

bool empilhar(Pilha *p, Item x);
bool desempilhar(Pilha *p, Item *x);
bool consultarTopo(Pilha *p, Item *x);

bool pilhaVazia(Pilha *p);
int tamanhoPilha(Pilha *p);

#endif
```

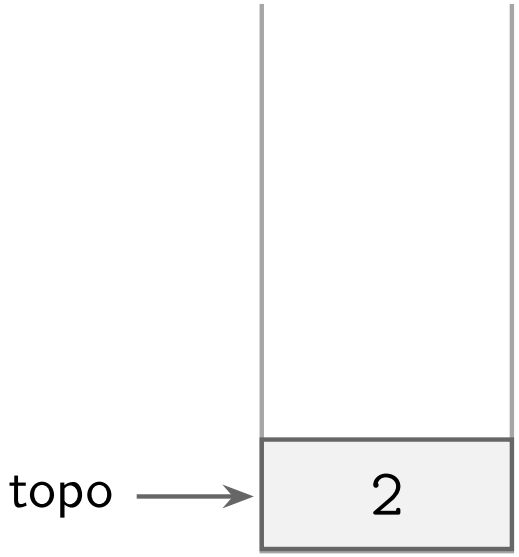
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `Pilha *p = criarPilha();`



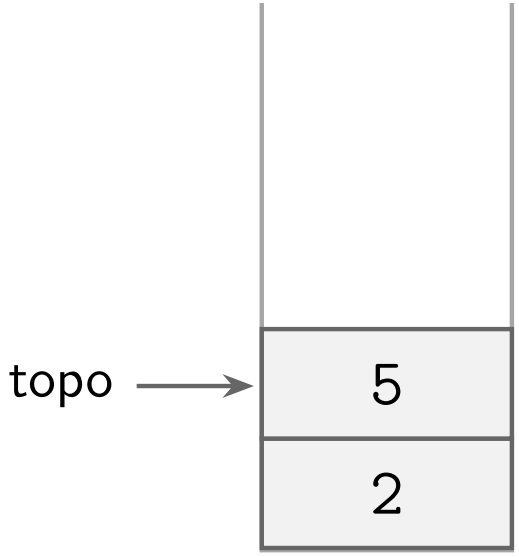
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `2` ⇒ Operação: `empilhar(p, 2)`



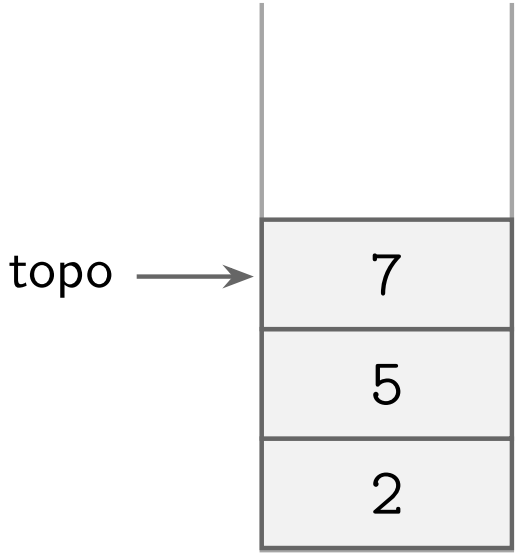
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `5` ⇒ Operação: `empilhar(p, 5)`



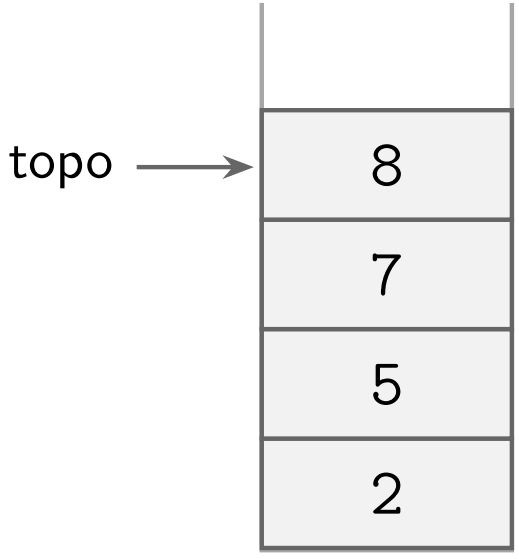
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `7` ⇒ Operação: `empilhar(p, 7)`



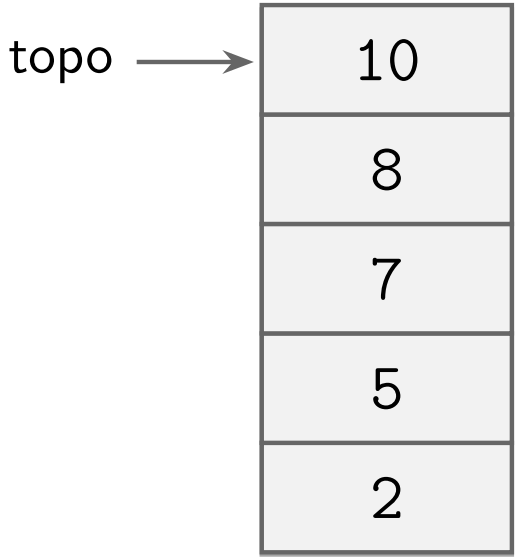
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `8` ⇒ Operação: `empilhar(p, 8)`



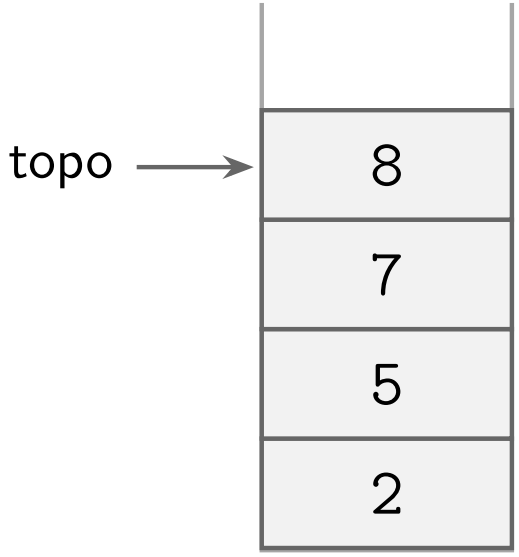
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Lido: `10` ⇒ Operação: `empilhar(p, 10)`



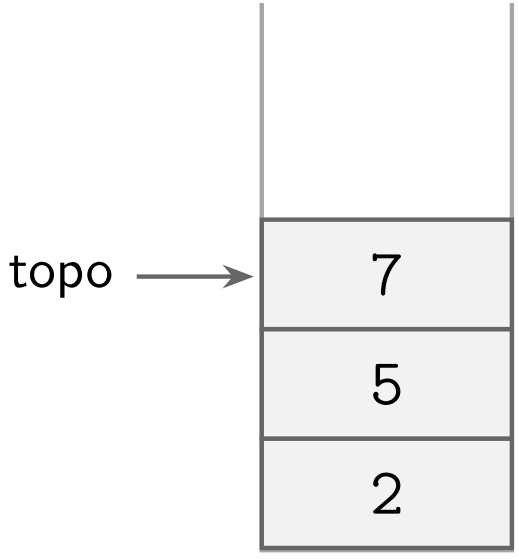
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10`



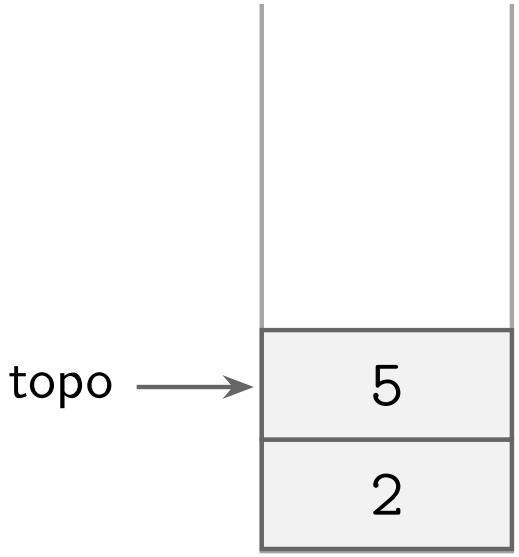
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8`



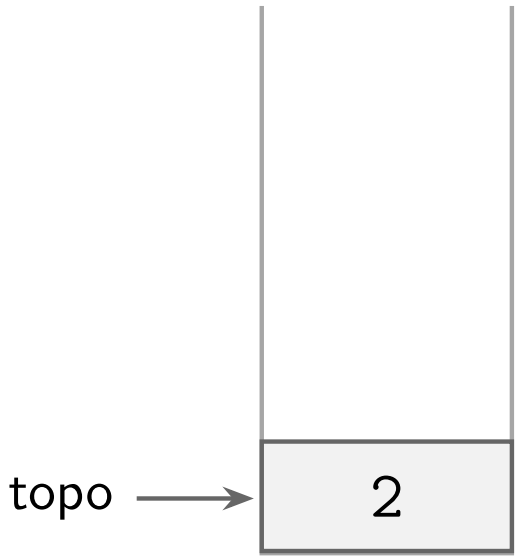
Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8 7`



Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8 7 5`



Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `desempilhar(p, &item)` ⇒ Saída: `10 8 7 5 2`



Exemplo: Inversão de uma Sequência

- **Problema:** inverter a ordem de uma sequência de valores.
- Entrada: `2 5 7 8 10`
- Operação: `destruirPilha(p)` $\Rightarrow$ A pilha é liberada.

Exemplo: Inversão de uma Sequência

```c
#include <stdio.h>
#include "pilha.h"

int main() {
    int entrada[5] = {2, 5, 7, 8, 10};
    Item item;

    Pilha *p = criarPilha();
    if (p == NULL)
        return 1;

    for (int i = 0; i < 5; i++)
        empilhar(p, entrada[i]);
    ...
```

Exemplo: Inversão de uma Sequência

```
    ...
    while (desempilhar(p, &item))
        printf("%d ", item);

    destruirPilha(p);
    return 0;
}
```

Implementação Sequencial

Implementação Sequencial

- Na implementação sequencial, os elementos são guardados em um vetor de capacidade **fixa**.
- Um campo inteiro registra a **posição** do elemento que está no topo.
- Empilhar **avança** essa posição e escreve no vetor; desempilhar **recua**.

Arquivo pilha_seq.c

Registro

```c
#include <stdlib.h>
#include "pilha.h"

#define MAX 100

struct pilha {
    Item itens[MAX];
    int topo;
};
```

Representando a Pilha Vazia

- O **campo** `topo` indica a posição do último elemento inserido no vetor.
- Com um elemento, `topo = 0`; com dois elementos, `topo = 1`...
- Para a pilha vazia, a **convenção** é `topo = -1`.
- O **número de elementos** é sempre `topo + 1`, inclusive quando a pilha está vazia.

Arquivo pilha_seq.c

Criação, Destruição e Consultas Simples

```
Pilha *criarPilha(void) {
    Pilha *p = (Pilha *) malloc(sizeof(Pilha));
    if (p != NULL)
        p->topo = -1;
    return p;
}

void destruirPilha(Pilha *p) { free(p); }

bool pilhaVazia(Pilha *p) { return p->topo == -1; }

int tamanhoPilha(Pilha *p) { return p->topo + 1; }
```

Arquivo `pilha_seq.c`

Inserção

```
bool empilhar(Pilha *p, Item x) {
    if (p->topo == MAX - 1)
        return false;

    p->topo++;
    p->itens[p->topo] = x;
    return true;
}
```

- O teste inicial verifica se o vetor já está **cheio**.

Arquivo `pilha_seq.c`

Remoção

```
bool desempilhar(Pilha *p, Item *x) {
    if (p->topo == -1)
        return false;

    *x = p->itens[p->topo];
    p->topo--;
    return true;
}
```

- O elemento é **copiado** para o endereço fornecido pelo usuário antes de o topo recuar.
- O valor permanece fisicamente no vetor, mas passa a estar **fora** da região válida.

Arquivo `pilha_seq.c`

Consulta ao Topo

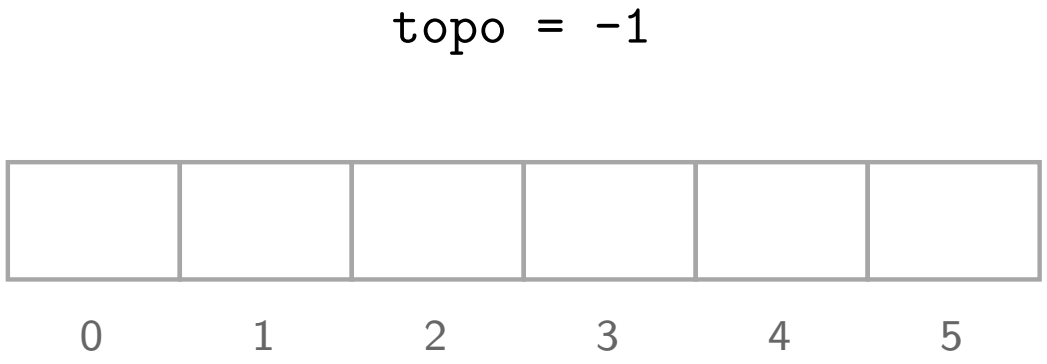
```
bool consultarTopo(Pilha *p, Item *x) {
    if (p->topo == -1)
        return false;

    *x = p->itens[p->topo];
    return true;
}
```

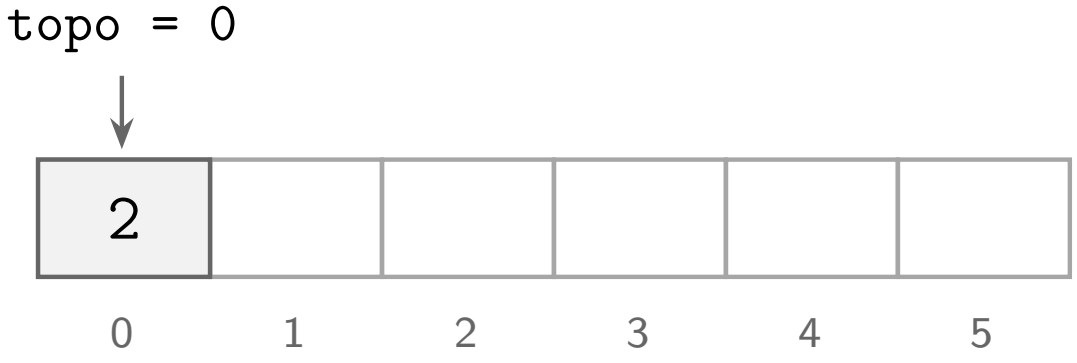
- Essa operação é útil quando o programa precisa decidir algo com base no topo antes de removê-lo.

Exemplo: Implementação Sequencial

1. `criarPilha()`

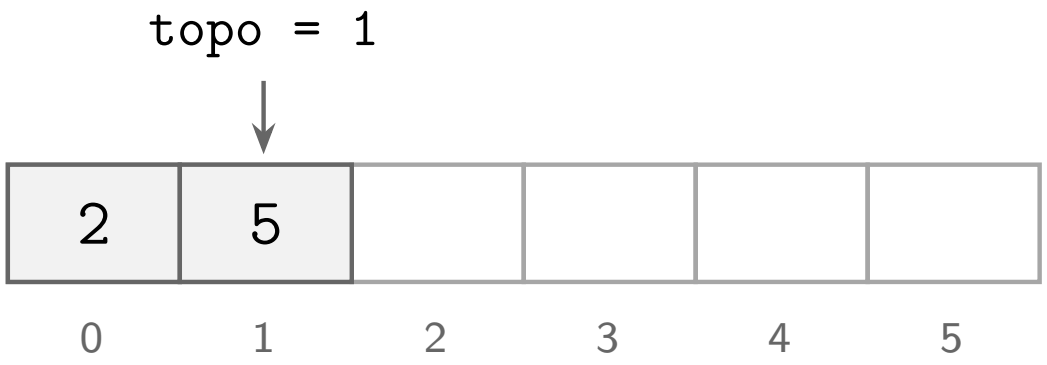


2. `empilhar(p, 2)`

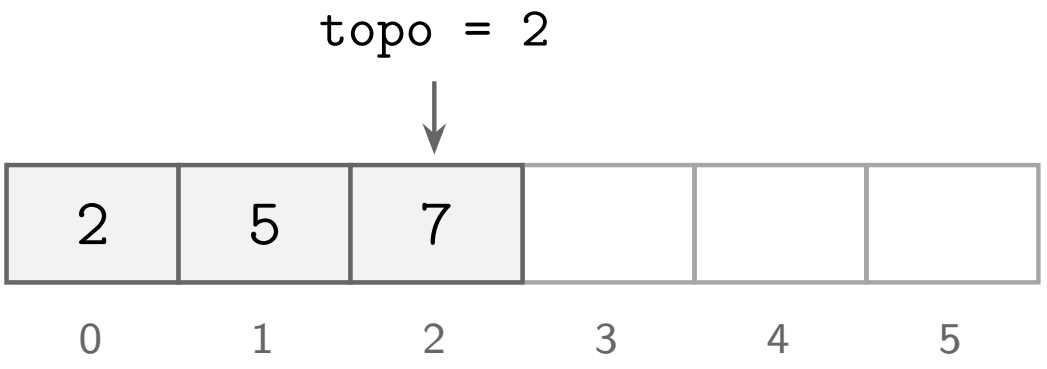


Exemplo: Implementação Sequencial

3. `empilhar(p, 5)`

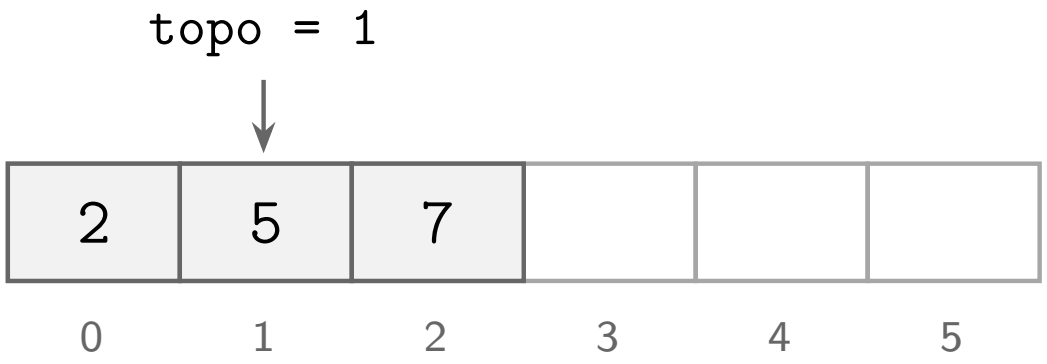


4. `empilhar(p, 7)`

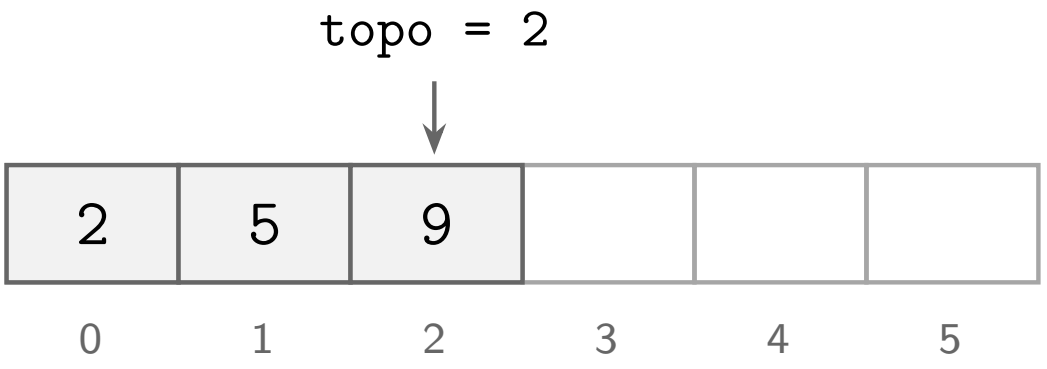


Exemplo: Implementação Sequencial

5. `desempilhar(p, &item)` devolve 7, e o topo recua.



6. `empilhar(p, 9)`



Custo das Operações

- Todas as operações executam um número fixo de comandos.
- Nenhuma delas contém laço, e nenhuma depende da quantidade de elementos armazenados.
- Dizemos que essas operações têm **custo constante**.

Limitações

- **Capacidade fixa:** a pilha não comporta mais do que `MAX` elementos.
- **Memória ociosa:** o vetor inteiro é reservado na criação, antes de saber quantos elementos serão empilhados.
- A alternativa é reservar memória a cada inserção, o que leva à **implementação encadeada**.

Implementação Encadeada

Implementação Encadeada

- Na implementação encadeada, a memória é reservada e devolvida conforme a necessidade, **um elemento por vez**.
- Cada elemento é armazenado em um bloco próprio, chamado de **nó**.
- Além do **valor**, cada nó guarda o **endereço** do nó que está abaixo dele na pilha.
- O registro da pilha guarda apenas o endereço do nó do **topo**.

Vantagens e Custo da Alocação sob Demanda

- Não existe capacidade máxima definida no código.
- Não há memória ociosa.
- Em compensação, cada nó carrega um ponteiro além do valor armazenado.
- Cada inserção passa a exigir uma chamada a `malloc()`, que não é gratuita.

Arquivo `pilha_enc.c`

Registros

```c
#include <stdlib.h>
#include "pilha.h"

typedef struct no {
    Item valor;
    struct no *prox;
} No;

struct pilha {
    No *topo;
    int n;
};
```

- O **campo `prox`** é um ponteiro para o próprio tipo que está sendo definido.

Arquivo pilha_enc.c

Criação e Consultas Simples

```
Pilha *criarPilha(void) {
    Pilha *p = (Pilha *) malloc(sizeof(Pilha));
    if (p != NULL) {
        p->topo = NULL;
        p->n = 0;
    }
    return p;
}

bool pilhaVazia(Pilha *p) { return p->topo == NULL; }

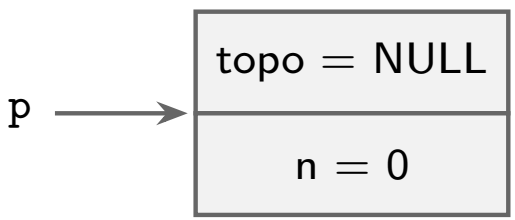
int tamanhoPilha(Pilha *p) { return p->n; }
```

- A pilha vazia é representada por `topo = NULL`.

Exemplo: Implementação Encadeada

Criação

- `criarPilha()` aloca apenas o registro de controle.



Arquivo pilha_enc.c

Inserção

```
bool empilhar(Pilha *p, Item x) {
    No *novo = (No *) malloc(sizeof(No));
    if (novo == NULL)
        return false;

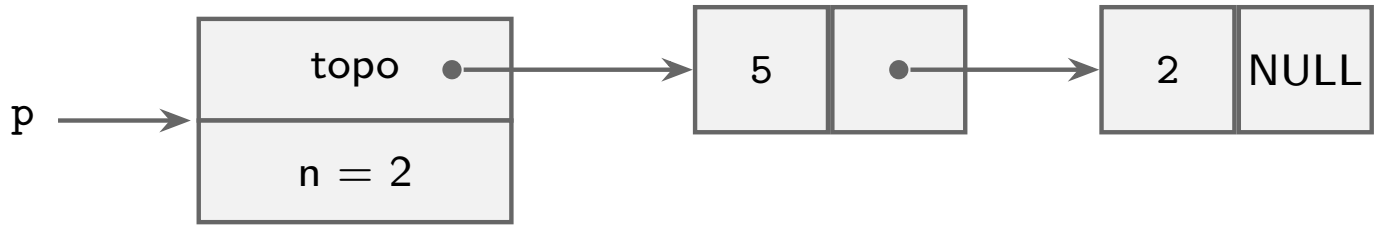
    novo->valor = x;
    novo->prox = p->topo;
    p->topo = novo;
    p->n++;
    return true;
}
```

- O nó é criado, preenchido e ligado ao antigo topo antes de o campo topo ser atualizado.

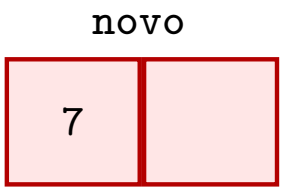
Exemplo: Implementação Encadeada

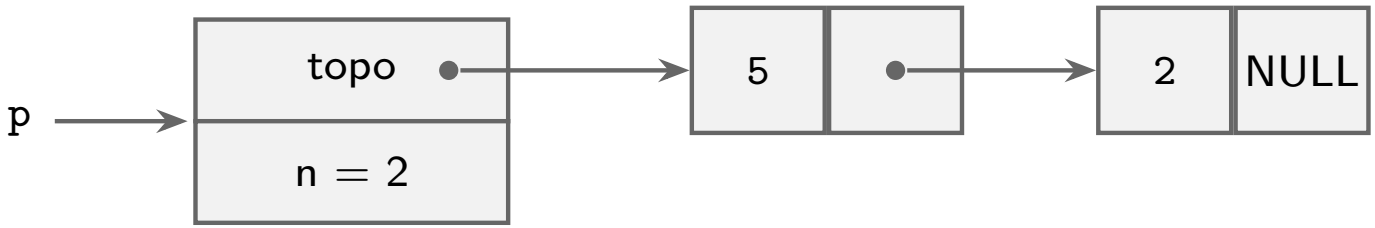
Inserção

1. Estado inicial, com dois elementos.



2. `No *novo = (No *) malloc(sizeof(No));`
   `novo->valor = x;`

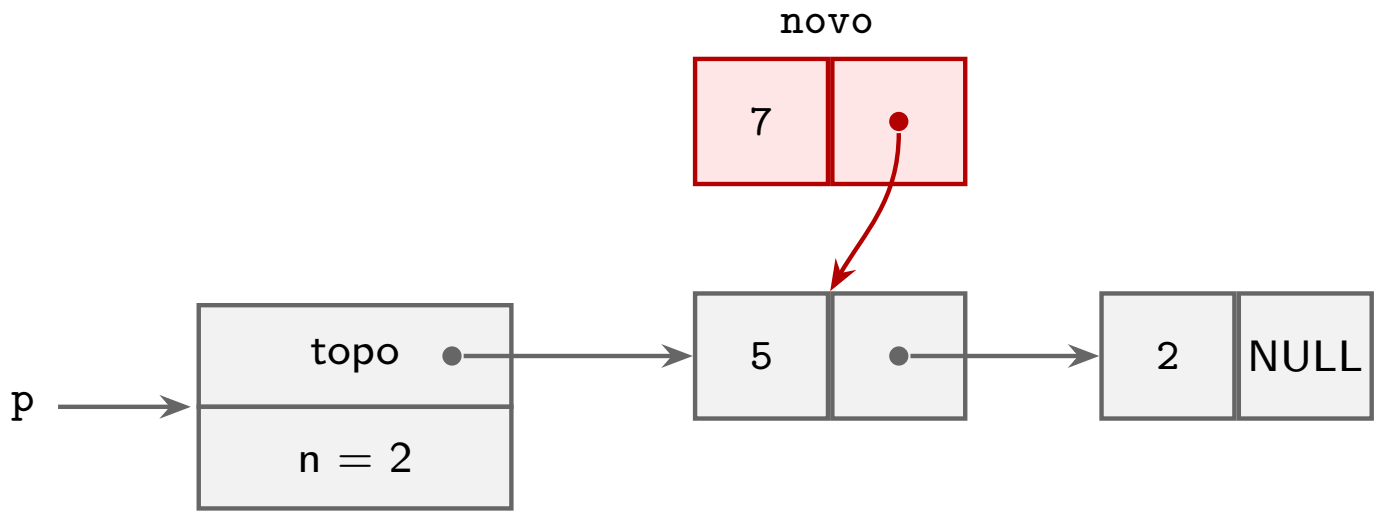




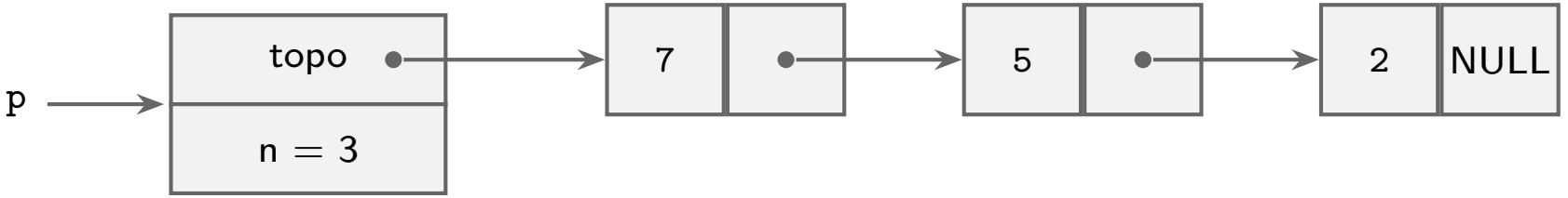
Exemplo: Implementação Encadeada

Inserção

3. `novo->prox = p->topo;`



4. `p->topo = novo;`
   `p->n++;`



Arquivo pilha_enc.c

Remoção

```
bool desempilhar(Pilha *p, Item *x) {
    if (p->topo == NULL)
        return false;

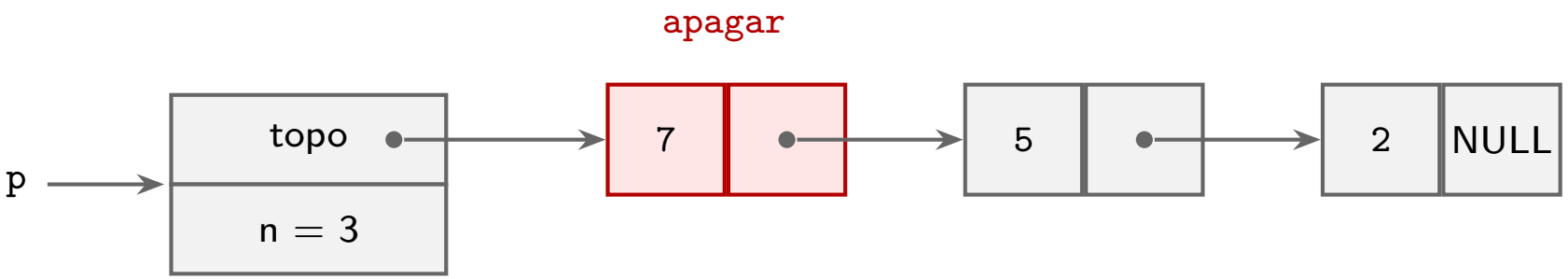
    No *apagar = p->topo;
    *x = apagar->valor;
    p->topo = apagar->prox;
    free(apagar);
    p->n--;
    return true;
}
```

- O endereço do nó é guardado em apagar antes de o topo avançar.

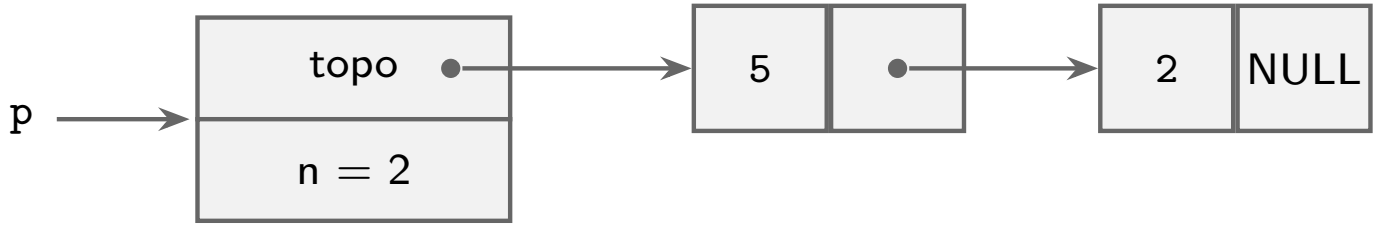
Exemplo: Implementação Encadeada

Remoção

5.
```
No *apagar = p->topo;
*x = apagar->valor; // *x = 7
```



6.
```
p->topo = apagar->prox;
free(apagar);
p->n--;
```



Arquivo `pilha_enc.c`

Consulta ao Topo

```
bool consultarTopo(Pilha *p, Item *x) {
    if (p->topo == NULL)
        return false;

    *x = p->topo->valor;
    return true;
}
```

Arquivo pilha_enc.c

Destruição

```
void destruirPilha(Pilha *p) {
    No *posicao = p->topo;
    No *apagar;

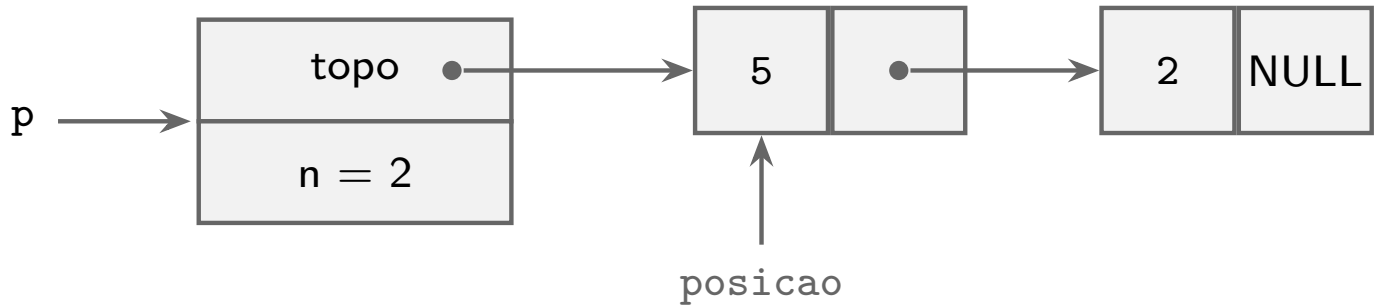
    while (posicao != NULL) {
        apagar = posicao;
        posicao = posicao->prox;
        free(apagar);
    }
    free(p);
}
```

- Cada nó precisa ser liberado individualmente.
- O ponteiro avança para o próximo nó antes de o atual ser liberado.
- O registro de controle é liberado por último.

Exemplo: Implementação Encadeada

Destruição

7. `No *posicao = p->topo;`

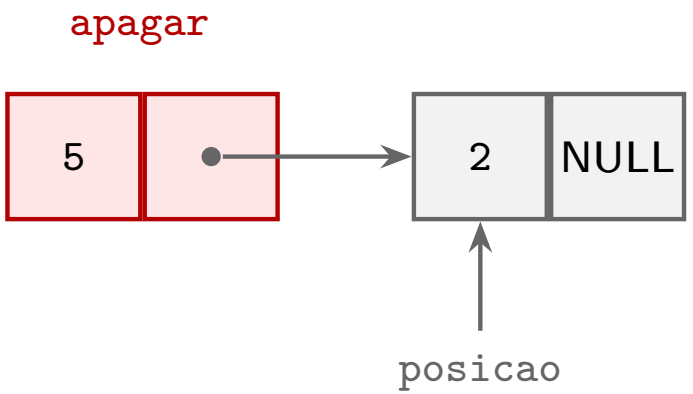


- As figuras seguintes mostram apenas os nós, porque é sobre eles que o laço age.
- O registro de controle só é liberado ao final.

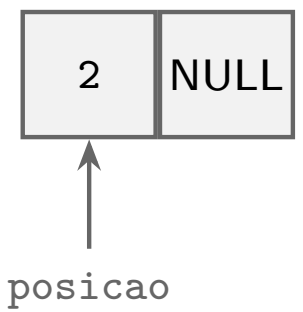
Exemplo: Implementação Encadeada

Destruição

8. `apagar = posicao;`
   `posicao = posicao->prox;`



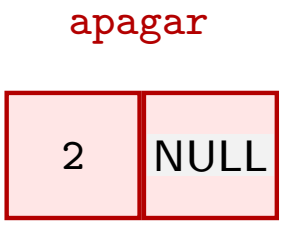
9. `free(apagar);`



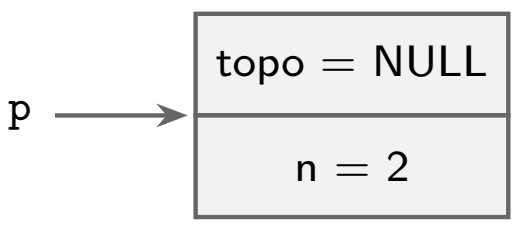
Exemplo: Implementação Encadeada

Destruição

10. `apagar = posicao;`
    `posicao = posicao->prox; // posicao = NULL`



11. `free(apagar); // O laço se encerra, e falta apenas liberar o registro de controle`



12. `free(p);`

Aplicações

Motivação

- Em ambas as aplicações a seguir, o texto é lido da esquerda para a direita e nem tudo pode ser resolvido de imediato.
- O que fica pendente precisa ser guardado e recuperado **na ordem inversa**.
- **Exemplo 1:** Em `(a + [b)]`, o delimitador aberto mais recentemente é o que deve ser fechado primeiro.
- **Exemplo 2:** Em `2 + 3 * 4`, a soma precisa esperar, porque o que vem depois altera o resultado.
- Precisamos de uma **pilha** nessas aplicações.

Delimitadores Balanceados

- Antes de resolver uma expressão aritmética, é preciso verificar se a expressão está **bem formada**.
- Uma expressão é **balanceada** quando cada delimitador de abertura tem um fechamento correspondente, na ordem correta.
- **Expressões balanceadas:**
  - `(a + b)`
  - `[a * (b + c)]`
  - `[a - (b)]`
- **Expressões não balanceadas:**
  - `(a + b`
  - `a + b)`
  - `(a + [b)]`

Algoritmo de Verificação

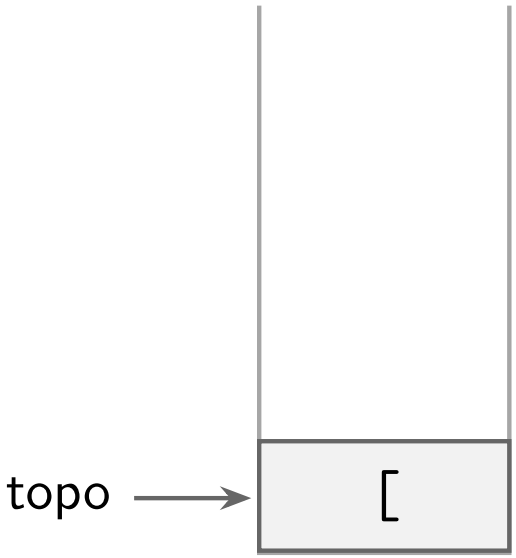
Ideia do Algoritmo

1. Criar uma pilha vazia.
2. Para cada caractere da expressão, da esquerda para a direita:
   a. Se for delimitador de abertura, empilhá-lo;
   b. Se for de fechamento, desempilhar um delimitador e verificar se ele combina com o encontrado;
   c. Caso contrário, ignorá-lo.
3. Ao final, verificar se a pilha está vazia.

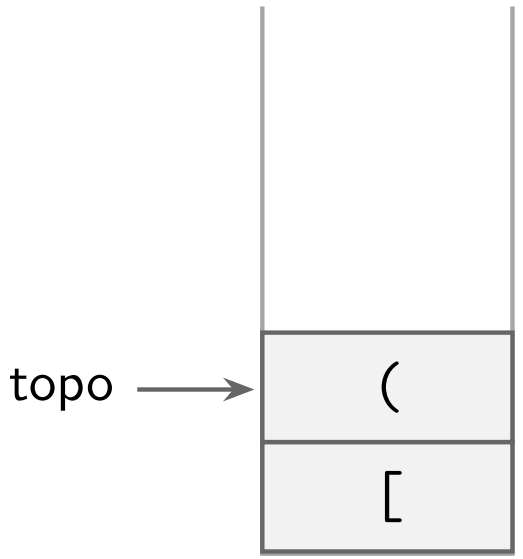
- A expressão é **rejeitada** em três situações: ao desempilhar de uma pilha vazia, quando os delimitadores não combinam, ou quando sobram elementos na pilha ao final.

Exemplo: Verificação de [a * (b + c)]

1. Lido [⇒ Empilhar

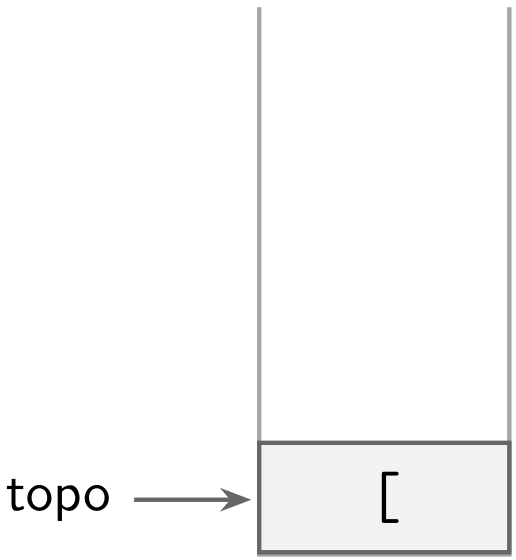


2. Lidos a e * ⇒ Ignorar
   Lido (⇒ Empilhar



Exemplo: Verificação de [a * (b + c)]

3. Lidos b, + e c ⇒ Ignorar
   Lido) ⇒ Desempilha (



4. Lido] ⇒ Desempilha [



5. A pilha termina vazia, então a expressão está **balanceada**.

Algoritmo de Verificação

```
bool combina(char abertura, char fechamento) {
    return (abertura == '(' && fechamento == ')')
        || (abertura == '[' && fechamento == ']')
        || (abertura == '{' && fechamento == '}');
}

bool balanceada(char *e) {
    Pilha *p = criarPilha();
    Item topo;

    for (int i = 0; e[i] != '\0'; i++) {
        if (e[i] == '(' || e[i] == '[' || e[i] == '{')
            empilhar(p, e[i]);
        ...
```

Algoritmo de Verificação

```
        ...
        else if (e[i] == ')' || e[i] == ']' || e[i] == '}')
        {
            if (!desempilhar(p, &topo) ||
                !combina(topo, e[i])) {
                destruirPilha(p);
                return false;
            }
        }
    }

    bool ok = pilhaVazia(p);
    destruirPilha(p);
    return ok;
}
```

Expressões Aritméticas

Existem três formas de escrever uma expressão aritmética:

- **Infixa:** o operador fica entre os operandos.
- **Pré-fixa (notação polonesa):** o operador vem antes.
- **Pós-fixa (notação polonesa reversa):** o operador vem depois.

Tabela 1: Exemplos de expressões aritméticas nas três notações.

Infixa	Pré-fixa	Pós-fixa
`A + B * C`	`+ A * B C`	`A B C * +`
`(A + B) * C`	`* + A B C`	`A B + C *`
`A + B - C`	`- + A B C`	`A B + C -`

Notação Pós-fixa

- Na notação **infixa**, saber qual operação executar primeiro exige consultar regras de precedência e parênteses.
- Em computação, a notação **pós-fixa** é a preferida porque pode ser avaliada na ordem de leitura.
- Calculadoras da HP, como a HP-12C, operam dessa forma até hoje; e compiladores convertem expressões para essa notação antes de gerar código.
- O tratamento completo de uma expressão tem, portanto, **duas etapas**:
  1. Converter da notação infixa para pós-fixa;
  2. Avaliar a expressão.

Avaliando uma Expressão Pós-fixa

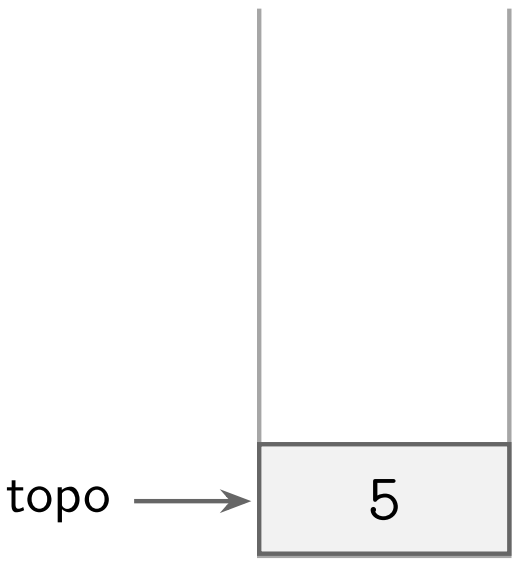
Ideia do Algoritmo

1. Criar uma pilha vazia.
2. Para cada símbolo da expressão, da esquerda para a direita:
   a. Se for um operando, empilhá-lo;
   b. Se for um operador, desempilhar dois valores, aplicar a operação e empilhar o resultado.
3. Ao final, desempilhar o único valor restante: o resultado.

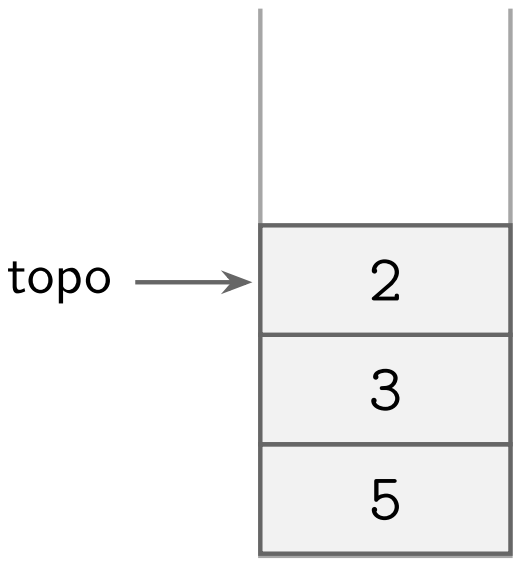
- **Observação:** o primeiro valor desempilhado é o operando da direita.

Exemplo: Avaliação de 5 3 2 * + 4 -

1. Lido 5 ⇒ Empilhar

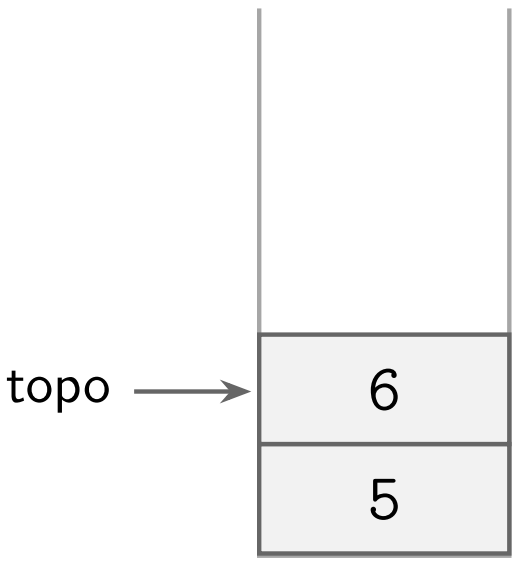


2. Lidos 3 e 2 ⇒ Empilhar

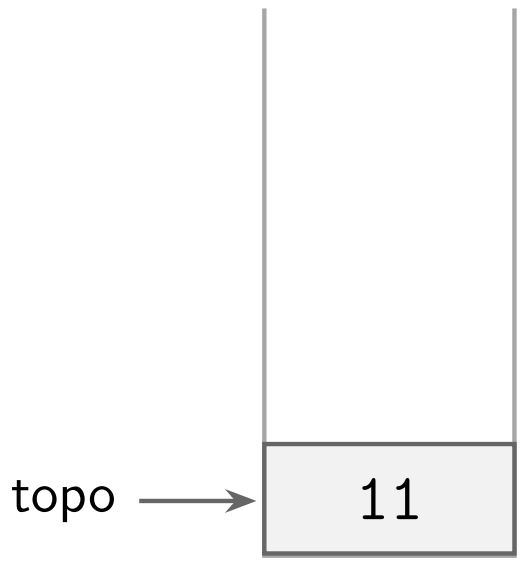


Exemplo: Avaliação de 5 3 2 * + 4 -

3. Lido * ⇒ Desempilha 2 e 3, empilha 6

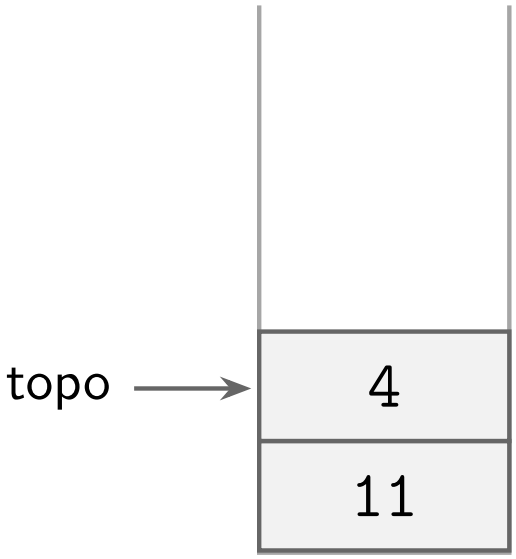


4. Lido + ⇒ Desempilha 6 e 5, empilha 11

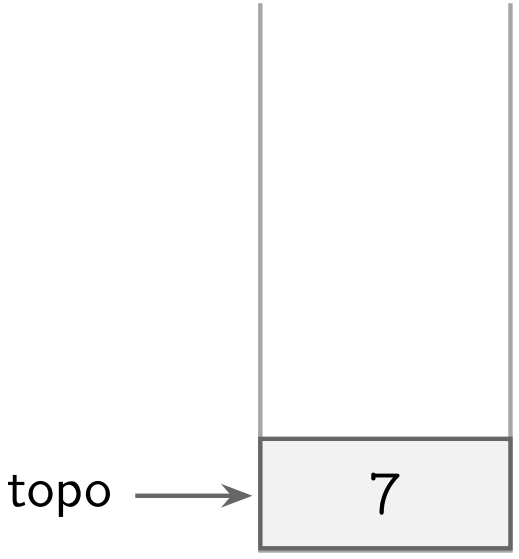


Exemplo: Avaliação de 5 3 2 * + 4 -

5. Lido 4 ⇒ Empilhar



6. Lido - ⇒ Desempilha 4 e 11, empilha 7



7. Desempilha o único valor restante: o resultado é 7.

Avaliando uma Expressão Pós-fixa

```
int calcular(char op, int a, int b) {
    switch (op) {
        case '+':
            return a + b;
        case '-':
            return a - b;
        case '*':
            return a * b;
        case '/':
            return a / b;
    }
    return 0;
}
```

Avaliando uma Expressão Pós-fixa

```
int avaliarPosFixa(char *e) {
    Pilha *p = criarPilha();
    Item a, b, r;
    for (int i = 0; e[i] != '\0'; i++) {
        if (e[i] >= '0' && e[i] <= '9')
            empilhar(p, e[i] - '0');
            // Empilha o valor numérico
        else if (e[i] != ' ') {
            desempilhar(p, &b);
            desempilhar(p, &a);
            empilhar(p, calcular(e[i], a, b));
        }
    }
    desempilhar(p, &r);
    destruirPilha(p);
    return r;
}
```

Convertendo de Infixa para Pós-fixa

- Na **conversão** de infixa para pós-fixa, a ordem dos operandos é a mesma; apenas os operadores mudam de lugar.
- Cada operador precisa esperar até que se saiba se o próximo tem precedência maior.
- A pilha guarda os operadores pendentes.

Algoritmo de Conversão

Ideia do Algoritmo

1. Criar uma pilha vazia e uma saída vazia.
2. Para cada símbolo da expressão infixa, da esquerda para a direita:
   a. Se for operando, enviar direto para a saída;
   b. Se for (, empilhar;
   c. Se for), desempilhar para a saída até encontrar (, que é descartado;
   d. Se for operador, enquanto o topo for um operador de precedência maior ou igual, desempilhá-lo para a saída; em seguida, empilhar o operador atual.
3. Ao final, desempilhar todo o resto para a saída.

Exemplo: Conversão de `A + B * C - D`

Símbolo	**Ação**	**Pilha**	**Saída**
`A`	Envia para a saída	vazia	`A`
`+`	Pilha vazia, empilha	`+`	`A`
`B`	Envia para a saída	`+`	`A B`
`*`	Topo tem precedência menor, empilha	`+ *`	`A B`
`C`	Envia para a saída	`+ *`	`A B C`
`-`	Desempilha * e +, empilha -	`-`	`A B C * +`
`D`	Envia para a saída	`-`	`A B C * + D`
Fim	Desempilha -	vazia	`A B C * + D -`

Exemplo: Conversão de (A + B) * C

Símbolo	**Ação**	**Pilha**	**Saída**
(	Empilha	(	vazia
A	Envia para a saída	(	A
+	Topo é (, empilha	(+	A
B	Envia para a saída	(+	A B
)	Desempilha até (, descarta (	vazia	A B +
*	Pilha vazia, empilha	*	A B +
C	Envia para a saída	*	A B + C
Fim	Desempilha *	vazia	A B + C *

Algoritmo de Conversão

```
int precedencia(char op) {
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return 0;
}

void infixaParaPosFixa(char *e, char *saida) {
    Pilha *p = criarPilha();
    Item topo;
    int k = 0;

    for (int i = 0; e[i] != '\0'; i++) {
        if (e[i] == '(')
            empilhar(p, e[i]);
        ...
```

Algoritmo de Conversão

```
...
else if (e[i] == ')') {
    while (desempilhar(p, &topo) && topo != '(') {
        saida[k] = topo;
        k++;
    }
}
else if (precedencia(e[i]) > 0) {
    while (consultarTopo(p, &topo) && topo != '('
        && precedencia(topo) >= precedencia(e[i])) {
        desempilhar(p, &topo);
        saida[k] = topo;
        k++;
    }
    empilhar(p, e[i]);
}
...
```

Algoritmo de Conversão

```
        ...
        else if (e[i] != ' ') {
            saida[k] = e[i];
            k++;
        }
    }

    while (desempilhar(p, &topo)) {
        saida[k] = topo;
        k++;
    }

    saida[k] = '\0';
    destruirPilha(p);
}
```